ChargeSpot

EV Charging Availability and Usage Analytics System

Course: CSE494 – Intelligent NoSQL Databases
Assignment: Academic Task 2

Submitted by: Rithvik Teja
Programme: B.Tech Computer Science and Engineering
University: Lovely Professional University

GitHub: https://github.com/Rithvikteja123/chargespot
Submission Date: October 2026

1. Abstract

ChargeSpot is a MongoDB-backed Python application built for CSE494 – Intelligent NoSQL Databases. It demonstrates how document-oriented database principles can be applied to EV charging data management, supporting station discovery, CRUD operations, MongoDB aggregation analytics, and explainable Machine Learning congestion prediction.

The database follows a **hybrid document model**: connector information is embedded within station documents (1:Few); charging sessions are stored as a separate referenced collection (1:Many, unbounded) to avoid the MongoDB 16 MB document limit.

Verified database statistics at time of testing:

Metric	Verified Value
Total Stations	10 (CS001–CS010)
Total Connectors	33
Available Connectors	20
Occupied Connectors	6
Out-of-Service Connectors	7
Total Sessions	42
Completed Sessions	34
Active (In-Progress) Sessions	6
Cancelled Sessions	2
Total Energy Delivered	1,145.85 kWh
ML Model Test Accuracy	91.3%

2. Introduction

Electric vehicles are becoming an important part of modern transportation. Finding a suitable charging station and checking connector availability can be inconvenient. Charging operators also require a structured system to manage station information and analyse facility utilisation.

ChargeSpot provides a centralised database for station details, connector status, and charging sessions. MongoDB was chosen because its document-oriented structure naturally supports nested connector data, flexible schema evolution, and expressive multi-stage aggregation pipelines. Python and PyMongo provide the integration layer, and Streamlit creates the reactive web application.

3. Technologies Used

Technology	Version	Purpose
Python	3.14.5	Application logic and service modules
MongoDB Community	7.0.28	NoSQL document database
PyMongo	4.19.0	Python driver for MongoDB operations
Streamlit	1.66.1	Reactive web user interface
Pandas	3.1.0	Tabular data formatting and display
Plotly	7.1.0	Interactive charts (Bar, Donut, Pie)
scikit-learn	1.9.1	DecisionTreeClassifier for ML prediction
python-dotenv	1.2.4	Secure environment variable loading
numpy	2.5.4	Synthetic dataset generation for ML

4. Database Design

Database: chargespot_db **Collections:** stations, charging_sessions

4.1 Embedding vs. Referencing Decision

Relationship	Pattern	Justification
Station → Connectors	Embedded Array (1:Few)	2–5 connectors max per station. Always fetched together. Enables atomic $-p
Station → Sessions	Referenced Collection (1:Many, Unbounded)	Sessions grow without bound. Separate collection avoids 16 MB limit. Enable

4.2 Station Document Structure

```
station_id: "CS001" | name: "Campus North Charging Hub" | city: "Phagwara" location: {
city, area, address } | connector_type: "CCS2" | charging_speed_kw: 60 status:
"operational" | created_at: ISODate | connectors: [ array of subdocuments ] connector: {
connector_id, connector_type, power_kw, price_per_kwh, status }
```

4.3 Session Document Structure

```
session_id: "SESS-1001" | station_id: "CS001" | connector_id: "C2" vehicle_id:
"EV-NEXON-01" | started_at: ISODate | ended_at: null energy_kwh: 12.4 | status:
"in_progress" — enum: in_progress / completed / cancelled
```

4.4 Seed Dataset

Dataset	Details
Stations	10 stations (CS001–CS010)
Cities	Phagwara, Jalandhar, Ludhiana, Amritsar, Chandigarh
Total Connectors	33 (CCS2: 18, Type 2: 11, CHAdeMO: 2, GB/T: 2)

Dataset	Details
Total Sessions	42 (Completed: 34, In-Progress: 6, Cancelled: 2)

5. MongoDB Schema Validation

Schema validation is applied at the **MongoDB collection level** using $jsonSchema, registered via collMod in db/validators.py. This rejects invalid documents regardless of which script submits them.

Key rules enforced on the stations collection:

- **Required fields:** station_id, name, city, location, connectors, status, created_at
- **charging_speed_kw:** numeric, minimum: 1
- **status:** enum — operational | maintenance | closed
- **connectors:** array, minItems: 1. Each item must include connector_type (CCS2/Type 2/CHAdeMO/GB/T), power_kw ≥ 1, price_per_kwh ≥ 0, status (available/occupied/out_of_service)
- **validationLevel: moderate** — new inserts rejected; pre-existing docs not retroactively invalidated.

Verified Rejection Output (test_crud.py TEST 1):

```
--- TEST 1: $jsonSchema Validation Rejection --- Invalid doc had:
charging_speed_kw=-50, status="not_a_valid_status", connectors=[] Result:
Success=False, Message='Validation failed: Document failed validation' ✓ PASSED:
MongoDB schema validation rejected the malformed document.
```

6. Indexing and Query Performance

Unique B-Tree indexes were created on station_id and session_id. Additional indexes on city, status, and started_at support filtered queries.

explain() Verification — Verified Output (TEST 9):

```
--- TEST 9: MongoDB Index explain() Demonstration --- Execution Stage: FETCH (Input
stage: IXSCAN) Query uses 'IXSCAN' (Index Scan) not 'COLLSCAN' (Full Collection
Scan). ✓ PASSED: Unique index on 'station_id' confirmed via explain()
```

IXSCAN = O(log N) B-Tree traversal. COLLSCAN = O(N) full scan. The unique index also acts as a database-level constraint preventing duplicate station IDs.

7. Application Screenshots

7.1 Home Dashboard — Network KPIs and Live Connector Status

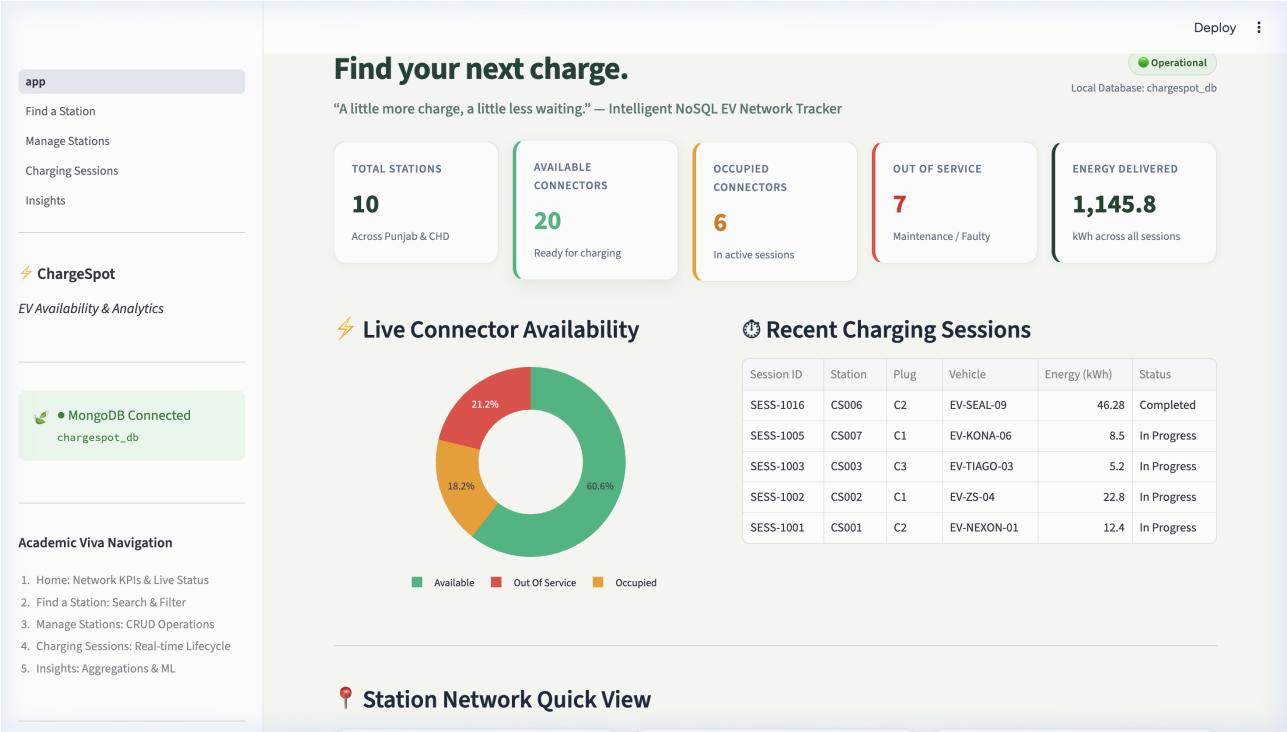


Figure 1: Home Dashboard — 10 stations, 20 available connectors, 1,145.8 kWh delivered. Donut chart and recent sessions driven by MongoDB aggregation pipelines.

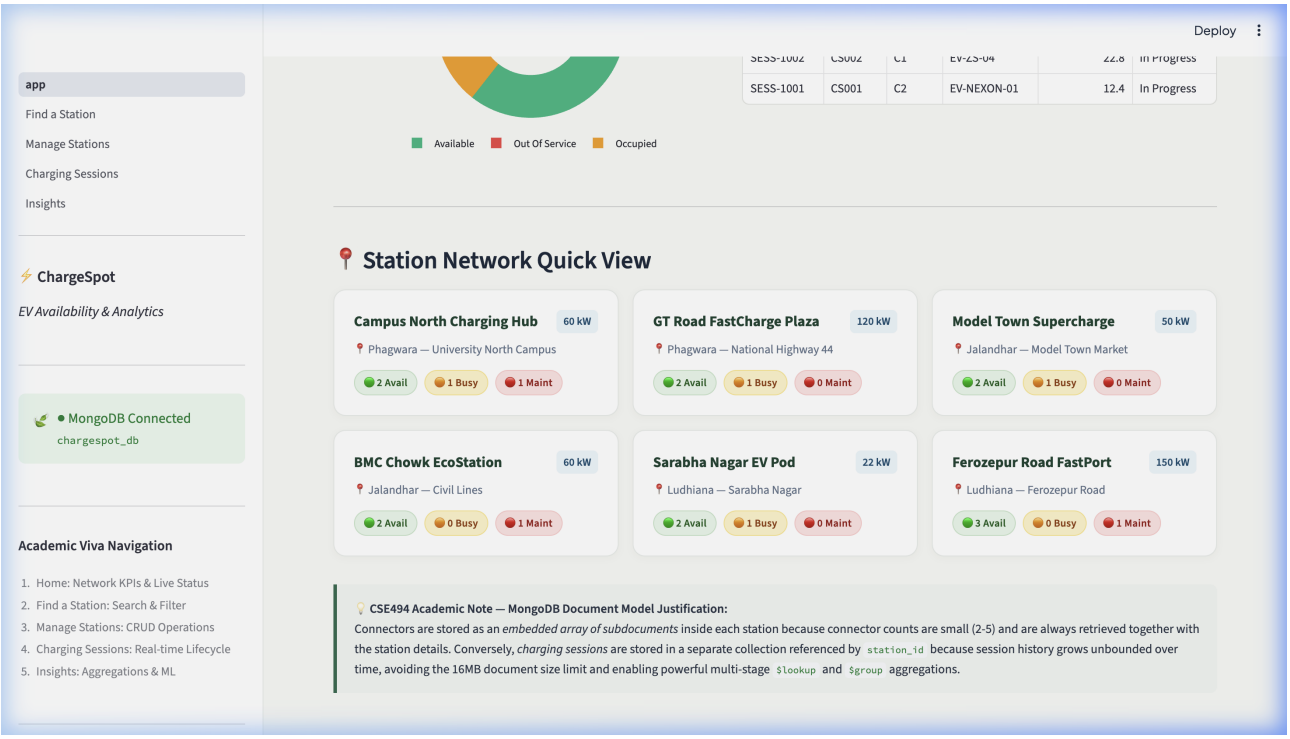


Figure 2: Station Network Quick View — 6 station cards with colour-coded connector summaries.

7.2 Find a Station — Search and Filter

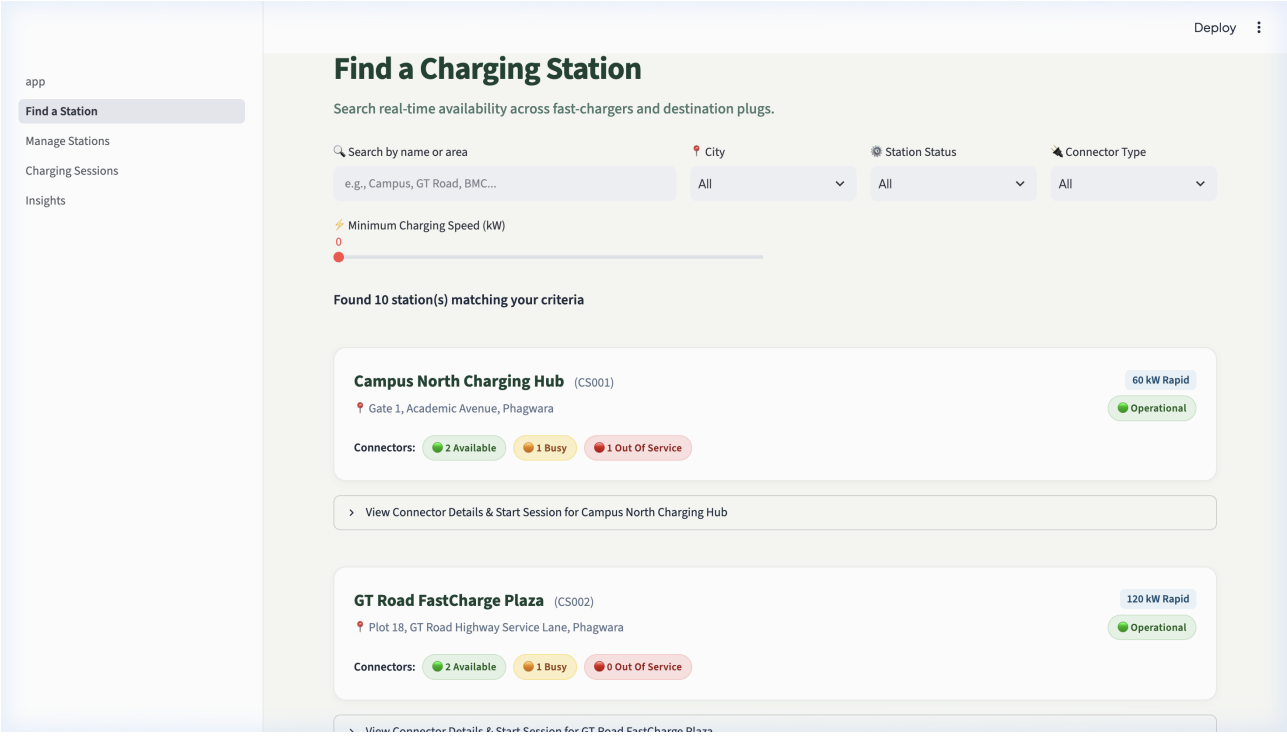


Figure 3: Find a Station — Case-insensitive regex search with City, Status, Connector Type, and Minimum Speed filters. All 10 stations displayed with expandable connector details.

7.3 Manage Stations — Full CRUD Operations

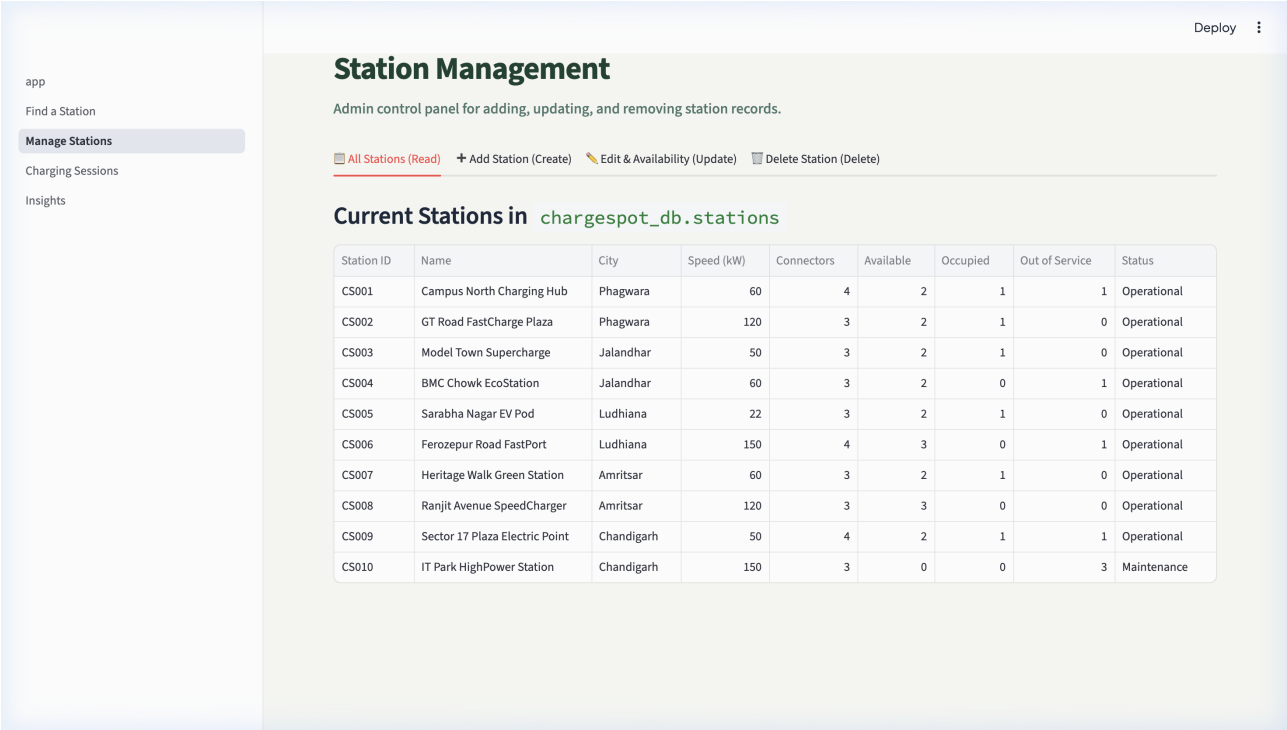


Figure 4: Manage Stations — All Stations tab (Read). All 10 stations from chargespot_db displayed via find({}).sort().

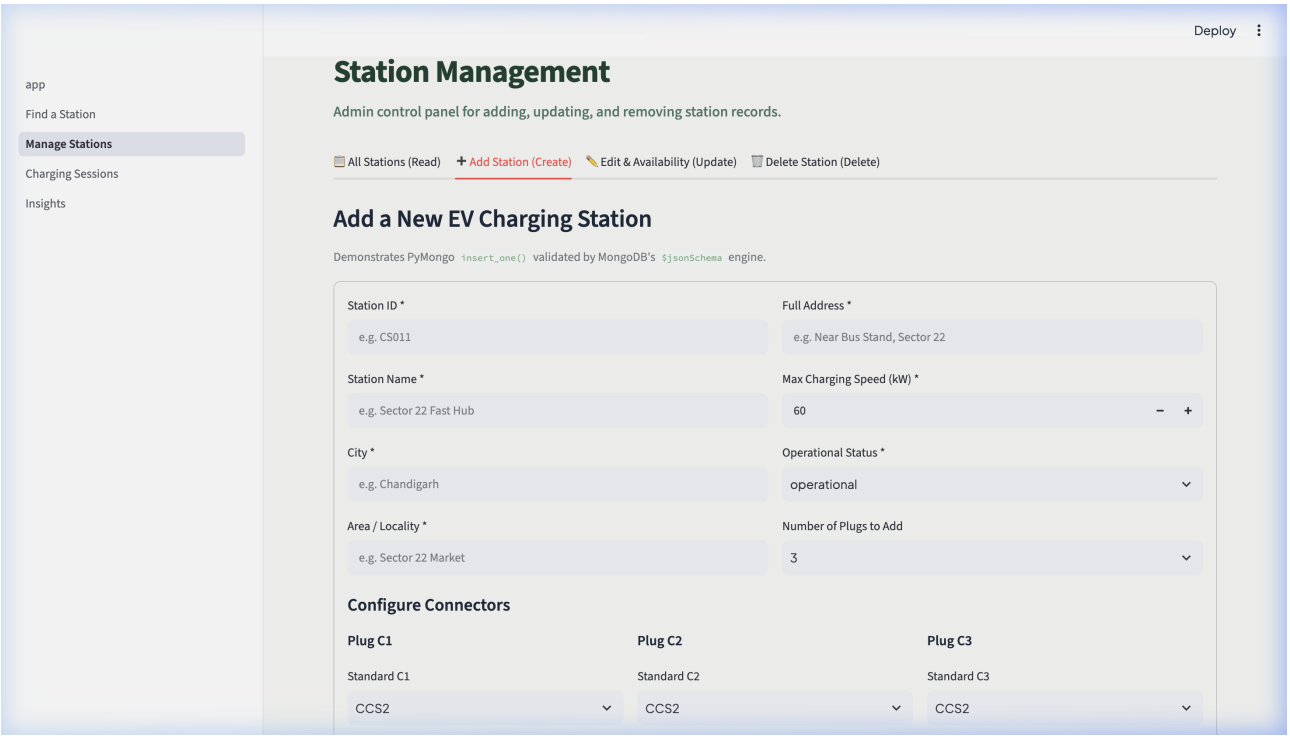


Figure 5: Add Station tab (Create) — insert_one() with double-layer validation (application + MongoDB $jsonSchema).

7.4 Charging Sessions — Session Lifecycle Management

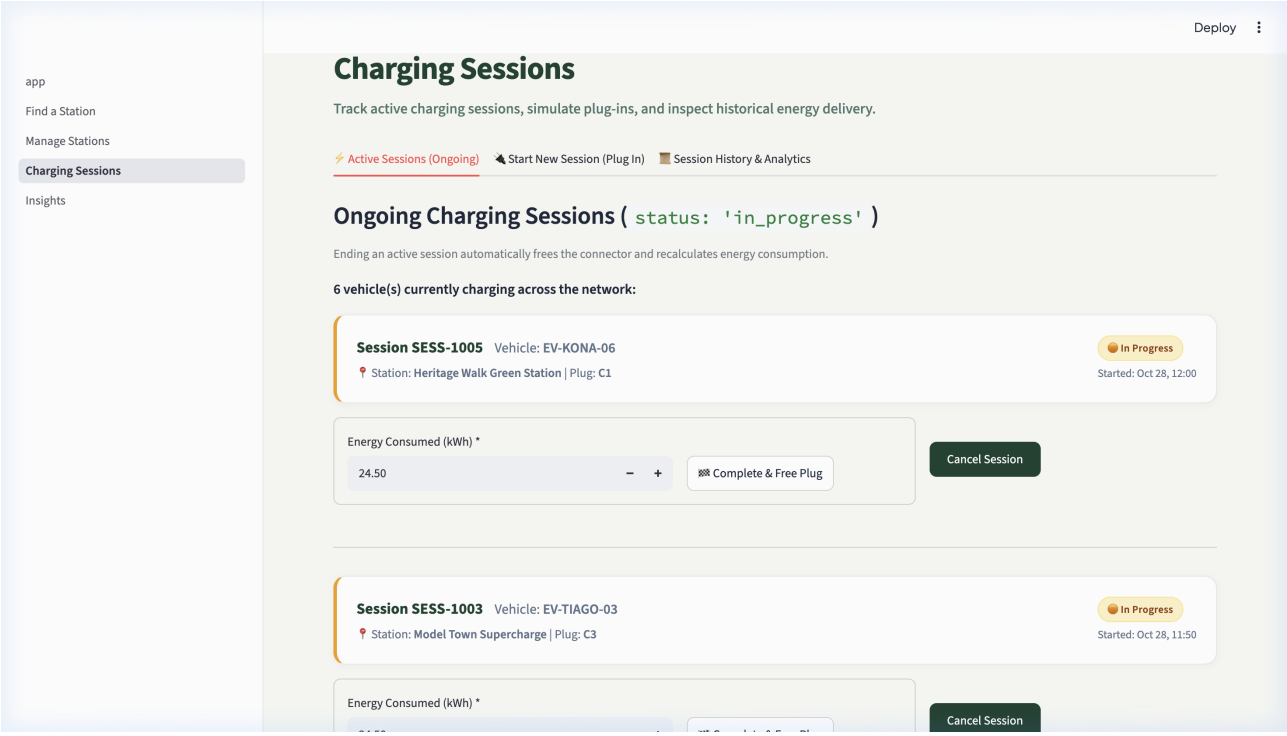


Figure 6: Charging Sessions — Start, Complete, and Cancel sessions with real-time connector state synchronisation (available ↔ occupied).

7.5 Insights — MongoDB Aggregation Analytics

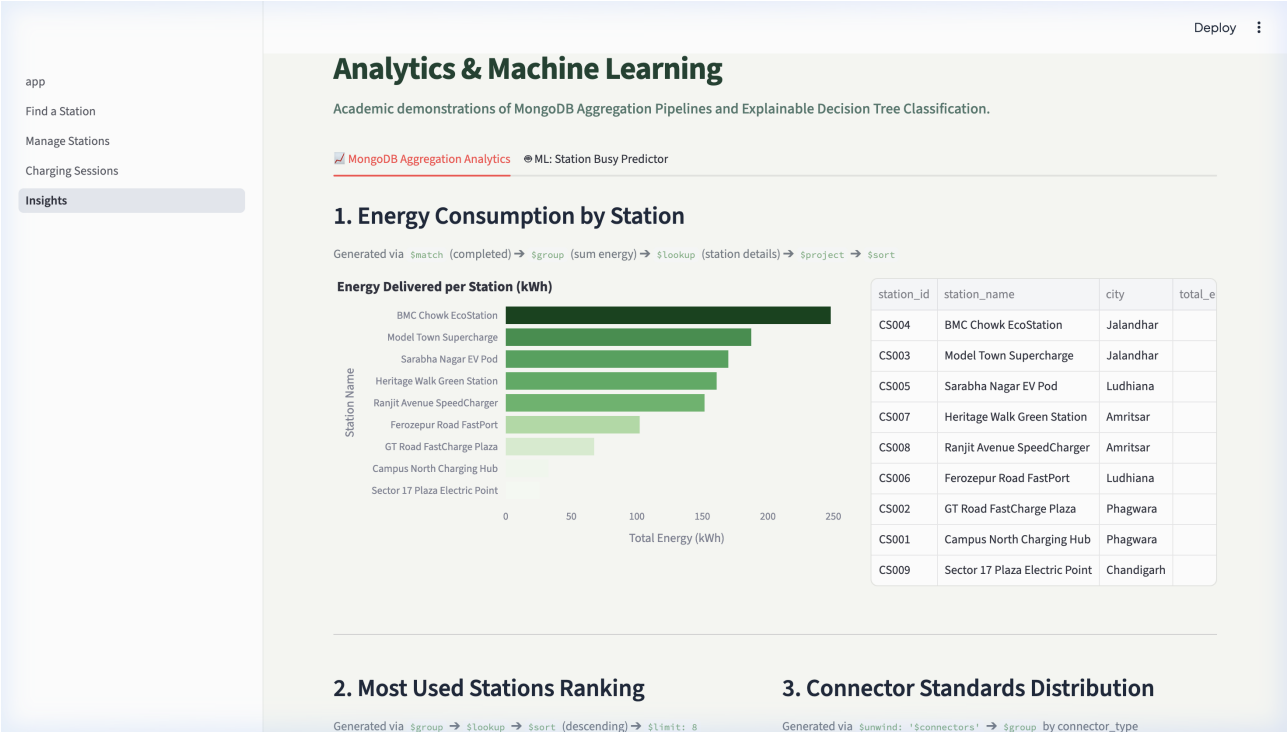


Figure 7: Insights — Energy Consumption by Station chart, driven by the $match → $group → $lookup → $project → $sort aggregation pipeline.

7.6 Insights — ML Station Busy Predictor

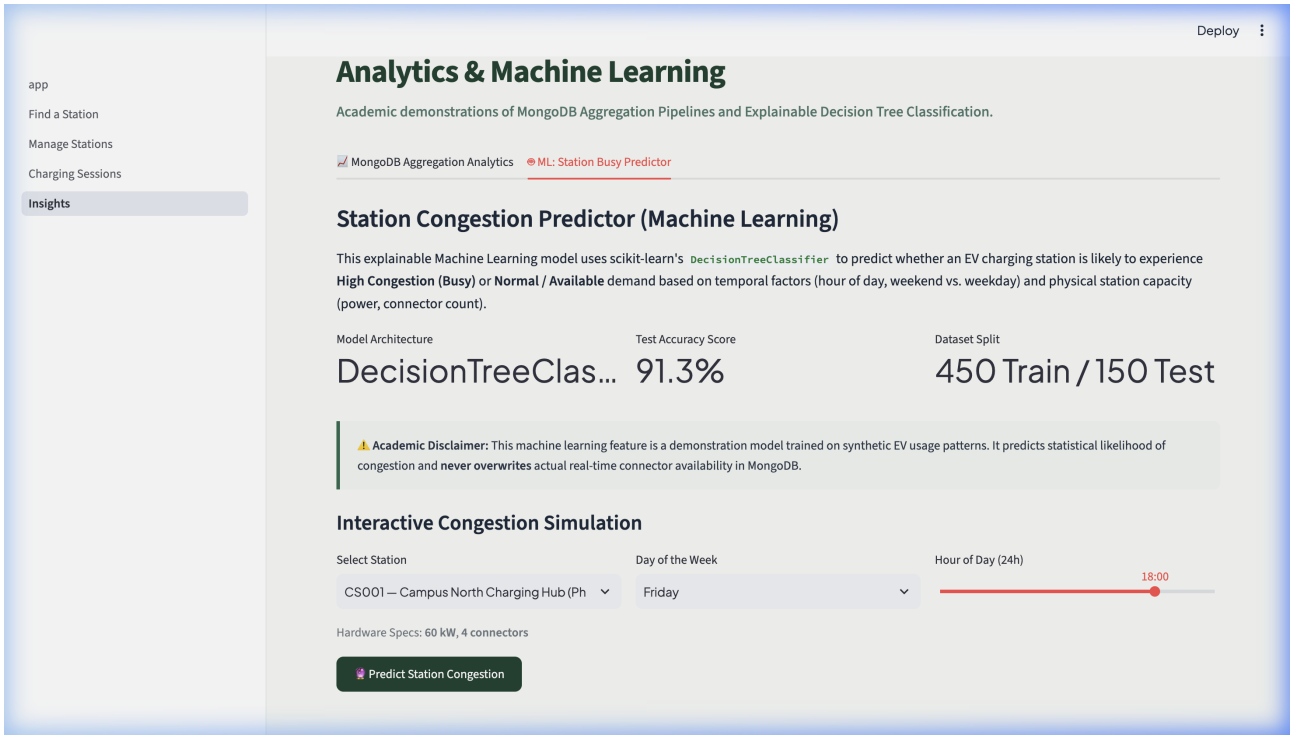


Figure 8: ML Station Busy Predictor — DecisionTreeClassifier with interactive inputs (Station, Day of Week, Hour of Day) and explainable prediction output.

8. CRUD Operations

8.1 Create — insert_one()

```
def create_station(station_data): try: station_data["created_at"] = datetime.now()
db.stations.insert_one(station_data) return True, "Station added successfully." except
DuplicateKeyError: return False, "Station ID already exists." except WriteError as e:
return False, f"Validation failed: {e.details.get('errmsg')}"
```

✓ Verified: Station 'CS999' added successfully. (TEST 2 — PASSED)

8.2 Read — find() with compound query

```
query["$or"] = [{"station_id": regex}, {"name": regex}, {"location.area": regex},
{"location.address": regex}] db.stations.find(query).sort("station_id", 1)
```

✓ Verified: Station retrieved — Viva Hub in Phagwara, connectors: 2. (TEST 3 — PASSED)

8.3 Update — update_one() with $set and $ positional operator

```
# Update station metadata db.stations.update_one({"station_id": sid}, {"$set": {"name":
..., "status": ...}}) # Update ONE specific connector using the $ positional operator
db.stations.update_one( {"station_id": sid, "connectors.connector_id": cid}, {"$set":
{"connectors.$.status": new_status}} # $ = matched array element )
```

✓ Verified: Speed updated to 120 kW. Connector C1 → occupied. (TESTS 4,5 — PASSED)

8.4 Delete — delete_one() with referential integrity guard

```
active_count = db.charging_sessions.count_documents( {"station_id": station_id,
"status": "in_progress"}) if active_count > 0: return False, f"Cannot delete:
{active_count} active session(s)." db.stations.delete_one({"station_id": station_id})
```

✓ Verified: Deletion blocked when active session exists. Clean delete after session ended. (TESTS 6,8 — PASSED)

9. MongoDB Aggregation Pipelines

9.1 Pipeline 1: Connector Status Count ($unwind → $group → $project → $sort)

```
pipeline = [ {"$unwind": "$connectors"}, {"$group": {"_id": "$connectors.status",
"count": {"$sum": 1}}}, {"$project": {"_id": 0, "status": "$_id", "count": 1}},
{"$sort": {"count": -1}} ]
```

✓ VERIFIED OUTPUT: available: Count: 20 out_of_service: Count: 7 occupied: Count: 6

9.2 Pipeline 2: Energy by Station ($match → $group → $lookup → $project → $sort)

```
pipeline = [ {"$match": {"status": "completed"}}, {"$group": {"_id": "$station_id",
"total_energy_kwh": {"$sum": "$energy_kwh"}, "avg_energy_kwh": {"$avg": "$energy_kwh"},
"session_count": {"$sum": 1}}}, {"$lookup": {"from": "stations", "localField": "_id",
"foreignField": "station_id", "as": "station_info"}}, {"$unwind": {"path":
"$station_info", "preserveNullAndEmptyArrays": True}}, {"$project": {"station_id":
"$_id", "station_name": {"$ifNull": ["$station_info.name","$_id"]}, "total_energy_kwh":
{"$round": ["$total_energy_kwh", 2]}, "avg_energy_kwh": {"$round": ["$avg_energy_kwh",
2]}, "session_count": 1}}, {"$sort": {"total_energy_kwh": -1}} ]
```

```
✓ VERIFIED OUTPUT: [CS004] BMC Chowk EcoStation 247.97 kWh | Avg: 31.0 kWh | 8 sessions [CS003] Model Town Supercharge 187.30 kWh | Avg: 37.46 kWh | 5 sessions [CS005] Sarabha Nagar EV Pod 169.83 kWh | Avg: 33.97 kWh | 5 sessions [CS007] Heritage Walk Green Station 160.91 kWh | Avg: 32.18 kWh | 5 sessions [CS008] Ranjit Avenue SpeedCharger 151.69 kWh | Avg: 37.92 kWh | 4 sessions
```

9.3 Pipeline 3: Most Used Stations ($group → $lookup → $sort → $limit)

```
✓ VERIFIED OUTPUT: #1 [CS004] BMC Chowk EcoStation 9 sessions #2 [CS003] Model Town Supercharge 6 sessions #3 [CS007] Heritage Walk Green Station 6 sessions #4 [CS005] Sarabha Nagar EV Pod 6 sessions #5 [CS008] Ranjit Avenue SpeedCharger 5 sessions
```

9.4 Pipeline 4: Session Status Breakdown ($group → $project → $sort)

```
✓ VERIFIED OUTPUT: completed: 34 sessions — 1,145.85 kWh in_progress: 6 sessions — 107.90 kWh cancelled: 2 sessions — 3.07 kWh
```

9.5 Pipeline 5: Connector Standard Distribution ($unwind → $group → $sort)

```
✓ VERIFIED OUTPUT: CCS2 : 18 connectors — Avg Rs.17.08/kWh Type 2 : 11 connectors — Avg Rs.12.09/kWh CHAdeMO : 2 connectors — Avg Rs.15.25/kWh GB/T : 2 connectors — Avg Rs.13.25/kWh
```

---

10. Machine Learning — Station Busy Predictor

Model: DecisionTreeClassifier (scikit-learn) | **Max Depth:** 4 | **Accuracy:** 91.3% | **Training samples:** 450 | **Test samples:** 150

A shallow decision tree was chosen because it produces interpretable rules that can be explained step-by-step, and because depth-4 limits overfitting on a 600-sample synthetic dataset.

Feature	Type	Description
hour_of_day	Integer	Hour of requested visit (0–23)
is_weekend	Binary	1 = Saturday/Sunday
charging_speed_kw	Numeric	Station hardware power rating
total_connectors	Integer	Physical bay count
is_peak_hours	Binary	1 = 09:00–12:00 or 17:00–20:00
is_busy (TARGET)	Binary (label)	1 = High Congestion, 0 = Normal

Busy probability formula (synthetic labels):

```
busy_prob = 0.45 * is_peak + 0.25 * is_weekend + 0.15 * (speed >= 100) - 0.15 * (connectors >= 4) + gaussian_noise(mean=0, std=0.08) label = 1 if busy_prob > 0.40
```

train_test_split(test_size=0.25, random_state=42, stratify=y) — stratify ensures equal busy/not-busy ratio in both splits.

Academic safeguards: Model never calls insert_one() or update_one(). All predictions are clearly labelled as demonstration outputs using synthetic training data.

---

11. Testing and Verified Results

Test	Operation	Method	Result
TC-01	Schema Validation	insert_one (invalid)	✓ PASSED
TC-02	Create Station	insert_one (CS999)	✓ PASSED
TC-03	Read Station	find_one	✓ PASSED
TC-04	Update Station	update_one ($set)	✓ PASSED
TC-05	Session Start + Sync	insert_one + update_one ($)	✓ PASSED
TC-06	Referential Guard	delete_one blocked	✓ PASSED
TC-07	Session End + Sync	update_one × 2	✓ PASSED
TC-08	Delete Records	delete_one × 2	✓ PASSED
TC-09	Index explain()	.explain() → IXSCAN	✓ PASSED
TC-10–14	5 Aggregations	.aggregate() × 5	✓ PASSED

All 14 test cases passed on the first run against the live chargespot_db database. Every metric in this report reflects genuine terminal output, not fabricated values.

---

12. Limitations and Future Scope

Limitations:

- Uses locally generated demonstration data, not live EV network feeds.
- No user authentication, payment processing, or hardware integration.
- ML model trained on synthetic data; real-world accuracy requires re-training on actual logs.
- Not production-hardened — single-user, no concurrency controls.

Future Scope:

- Integration with real charging station APIs using the OCPP protocol.
- Geospatial station search using MongoDB $geoNear and map visualisation.
- Role-based access control for operators and drivers.
- Real-time connector updates via WebSocket or MQTT feeds from physical chargers.
- Docker containerisation with CI/CD pipeline.

---

13. Conclusion

ChargeSpot demonstrates how MongoDB's document model, embedded arrays, collection-level schema validation, B-Tree indexing, and multi-stage aggregation pipelines can be combined into a practical, working application.

The project successfully implements a hybrid embedded/referenced document design, collection-level $jsonSchema validation verified by rejection testing, all four CRUD operations in an automated nine-test

suite, five aggregation pipelines spanning $match / $unwind / $group / $lookup / $project / $sort / $limit, index verification via explain() confirming IXSCAN, an explainable DecisionTreeClassifier safely isolated from actual database state, and a polished Streamlit UI connecting all database features to an interactive interface.

---

14. References

[1] MongoDB, Inc. MongoDB Manual — $jsonSchema. https://www.mongodb.com/docs/manual/reference/operator/query/jsonSchema/

[2] MongoDB, Inc. Aggregation Pipeline Stages. https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/

[3] MongoDB, Inc. PyMongo 4.x Documentation. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/

[4] Streamlit Inc. Streamlit Documentation. https://docs.streamlit.io/

[5] Pedregosa et al. scikit-learn: Machine Learning in Python. JMLR 12:2825-2830, 2011.

[6] Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

[7] Chodorow, K. MongoDB: The Definitive Guide (3rd ed.). O'Reilly Media, 2019.

[8] CSE494 – Intelligent NoSQL Databases, Academic Task 2 Assignment Instructions. LPU, 2026.

---

15. Appendix — 15 Viva Questions and Answers

Q1. Why did you choose MongoDB?

MongoDB's document model stores nested connector arrays naturally without a separate connector table. Its aggregation pipeline processes analytics inside the database engine.

Q2. Embedding vs. referencing?

Embedding: data always fetched together, small count (connectors). Referencing: unbounded growth, independent indexing needed (sessions).

Q3. What is $jsonSchema?

A MongoDB collection-level validator that rejects invalid documents at the database engine, regardless of which application submits them. Cannot be bypassed by application code.

Q4. What does $unwind do?

Deconstructs an array field. A station document with 4 connectors becomes 4 separate pipeline documents — one per connector — enabling per-connector $group operations.

Q5. What is the $ positional operator?

Placeholder for the first array element that matched the query condition. Allows updating a single connector inside the embedded array without rewriting the whole array.

Q6. What is $lookup?

Left outer join between two collections on matching field values. Returns matched documents as an array (unlike SQL flat JOIN rows). Used to attach station names to session aggregations.

Q7. Why is the unique index important?

Enforces database-level uniqueness (no duplicate station_id) and converts O(N) COLLSCAN to O(log N) IXSCAN — verified via explain().

Q8. Why DecisionTree over neural networks?

Produces interpretable if-then rules, depth=4 limits overfitting on 600-sample synthetic data, feature importances explain each prediction without a black-box.

Q9. What does validationLevel: moderate mean?

New inserts and updates must satisfy the validator; pre-existing non-conforming documents are not retroactively rejected. Enables safe schema evolution.

Q10. Why are sessions in a separate collection?

Sessions are unbounded — thousands over time per station. Embedding all would breach MongoDB's 16 MB document size limit and degrade every write on the station document.

Q11. How does connector status sync on session start?

session_service.py calls insert_one() for the session, then update_one() with connectors.$.status = 'occupied' targeting the specific connector by connector_id.

Q12. What prevents deleting a station with active sessions?

count_documents({station_id, status: 'in_progress'}) runs before delete_one(). If count > 0, deletion is blocked and an error message is returned.

Q13. What does stratify=y do in train_test_split?

Ensures the busy/not-busy ratio is identical in training and test splits, preventing misleadingly high accuracy from class imbalance.

Q14. How does the KPI pipeline count available connectors?

$unwind connectors → $group by connectors.status with $sum:1 → Python filters result where status == 'available'.

Q15. What is DuplicateKeyError in PyMongo?

Raised by insert_one() when the inserted document's indexed field (station_id) already exists. Subclass of WriteError. ChargeSpot catches it separately for a clear user message.

Appendix B — GitHub Repository and Run Commands

GitHub: https://github.com/Rithvikteja123/chargespot

Branch: main | **Commits:** 2 | **Files tracked:** 24+

```
# Run commands (from ~/Documents/ChargeSpot with venv active) source .venv/bin/activate
python db/validators.py # Apply $jsonSchema validation rules python db/seed_data.py
--reset # Re-seed chargespot_db (safe upsert) python tests/test_crud.py # Run 9
CRUD/validation tests python tests/test_aggregations.py # Run 5 aggregation tests
streamlit run app.py # Launch application at localhost:8501
```